Seventeenth Implementation Iteration: Request Lifecycle History

with Request Processing History Logging and Export

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this seventeenth implementation iteration is to improve the traceability of the prototype by adding lifecycle history logging to requests.

In previous iterations, the system was able to load requests from external CSV files, evaluate them through the rule engine, and export final request outcomes. However, each request only stored its final status and, in rejected cases, the rejection reason. The intermediate processing steps were not recorded inside the request itself.

In this iteration, each request now stores a simple lifecycle history log. The log records important events such as request creation, request evaluation, approval, rejection with reason, and export. This improves visibility into how each request moved through the system while preserving the current architecture and rule-based processing behavior.

2. Changes from Iteration 16

Compared with the sixteenth iteration, the following changes were introduced:

- Added lifecycle history storage to the `Request` base class
- Added support for appending textual lifecycle events during request processing
- Recorded a `created` event when a request object is constructed
- Recorded an `evaluated` event when the allocation service begins processing a request
- Recorded `approved` or `rejected` events depending on the processing result
- Preserved rejection reasons and included them as part of rejection history entries
- Extended `request_results.csv` export so that lifecycle history is included in readable form

3. Scenario

The prototype starts by loading users, spaces, and requests from external CSV files through the `DataController`. When each request object is created, its lifecycle history begins with a creation event. The allocation service then processes all requests and logs that evaluation has started before rule checking takes place.

If rule evaluation succeeds, the request is approved and allocations are created as before. If rule evaluation fails, the request is rejected and the rejection reason is still preserved. At the end of execution, the system exports:

- accepted allocations to `allocations.csv`
- full processed request outcomes to `request_results.csv`
- lifecycle history entries together with each exported request result

This means that the final exported request results now provide not only the final outcome, but also a compact record of how that outcome was reached.

4. Modules and Their Tasks

A. Request Model Module

- Store lifecycle history as part of the `Request` base class
- Provide simple support for adding lifecycle events
- Preserve existing request data such as requester, participant count, priority, feature, building, status, and rejection reason

B. Allocation Service Module

- Begin lifecycle tracking for processing by logging request evaluation
- Continue using the existing rule engine and facade without changing decision logic
- Mark requests as approved or rejected and update lifecycle history accordingly

C. Request Management Module

- Continue supporting both one-time and recurring requests through the existing inheritance structure
- Ensure lifecycle history applies uniformly to all request types through the shared base class

D. Output Export Module

- Continue exporting accepted allocations to `allocations.csv`
- Continue exporting request outcomes to `request_results.csv`
- Export the lifecycle history log in readable form together with each request result

E. Data Controller Module

- Continue serving as the centralized entry point for loading and exporting data
- Preserve the existing CSV-based workflow without structural redesign

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
13	Same	Same	Rule Engine + Facade
14	Same	Same	Structured user roles + role rule
15	Same	Same	Request priority model
16	Externalized requests	Class + Lab + Meet	DataController + CSV request loading
17	Same + history log	Class + Lab + Meet	Request lifecycle history logging + export

6. Updated Class Diagram

Figure 1 shows the updated class diagram including lifecycle history support in the `Request` model and the export of lifecycle data in the request results writer.

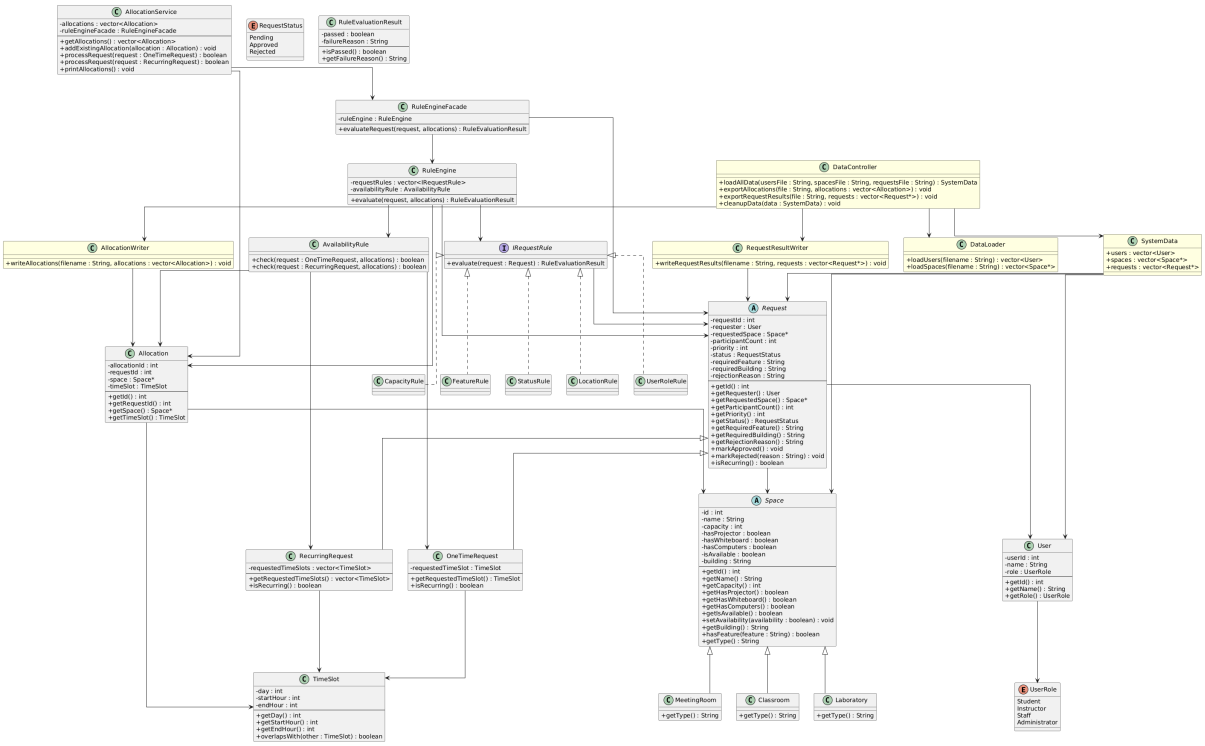

Figure 1: Updated Class Diagram with Request Lifecycle History

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. The system still loads data, evaluates requests, and exports results in the same overall sequence, but request lifecycle events are now recorded throughout the processing flow.

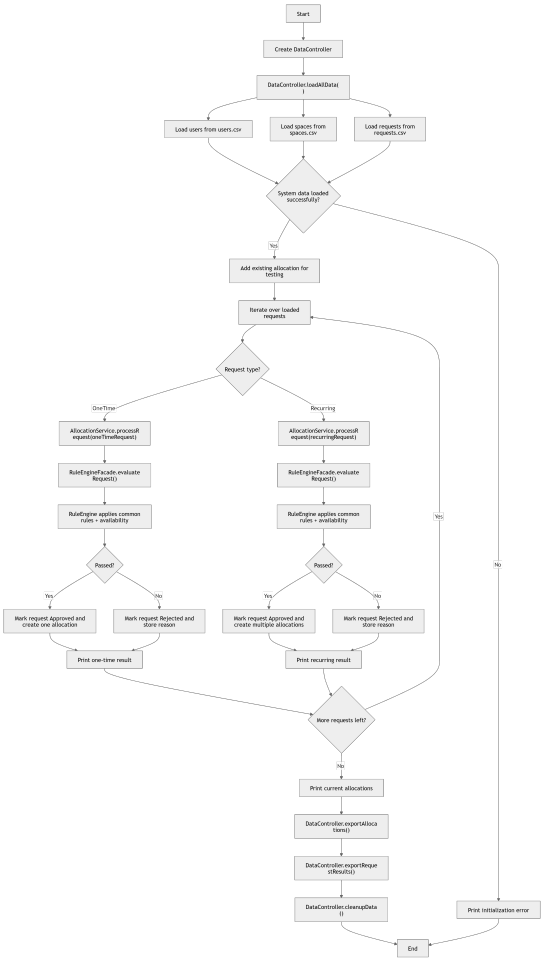

Figure 2: Workflow with Request Lifecycle History Logging

8. Prototype Implementation

The C++ prototype was extended by adding a simple lifecycle history log to the `Request` base class. This log stores textual processing events in sequence, which makes it possible to trace the path of each request from creation to final export.

The allocation service was updated so that it records when evaluation begins. The existing approval and rejection behavior was preserved, but lifecycle events are now appended when those outcomes occur. In rejected cases, the rejection reason remains available and is also reflected in the lifecycle history.

The request result export component was also extended so that the generated `request_results.csv`

file now includes the lifecycle history in readable form. This implementation remains conservative because it does not modify the rule decision logic, the CSV loading design, or the overall terminal-based prototype flow.

9. Summary

This seventeenth implementation iteration improves the traceability of the prototype by introducing request lifecycle history logging. Requests now keep a simple record of important events such as creation, evaluation, approval or rejection, and export.

This iteration does not change the allocation rules or the final decision logic of the system. Instead, it improves debugging, auditability, and demonstration of how each request was processed. As a result, the exported request results now provide a clearer and more complete view of the internal request processing flow.